

SEAL

PROTOCOL MULTISIG

v0.1

A pocket companion
for signers

feat. **THE RED GUILD**

Security Alliance

PROTOCOL

MULTISIG

v0.1

A pocket companion
for signers

Security Alliance

Table of Contents

About this book	9
How to use this book	10
What actually fails	13
The failure modes that matter	14
What this book is not	15
The job	16
Map the powers	17
Access control inventory	17
Two questions	18
Impact, not a dollar table	19
Write the decision down	21
Split the sets	22
Why split	22
Shapes, as examples	23
Treasury extra	25

On-chain beats policy	25
Delay	27
Tiered delays	28
The window is the control	29
Simulate the right thing	30
Emergency bypass	30
Who may sign	32
The floor	32
Independence	33
Proposer is not signer	34
Nested Safes	35
You can refuse	35
How you join	37
Fresh address	37
Prove ownership	38
Out of band	38

Channels before the first real tx	39
Bookmark the real Safe	40
Verify before you sign	41
Before anything else	42
Two phases (Safe / EVM)	43
Hash is not intent	44
Red flags. Stop	46
Address book	47
After you sign	47
Solana / Squads	48
When it breaks	49
Key compromise, first 30 minutes ..	49
Channel takeover	50
Backup UI	51
Pause	52
Drills	52
Say it clearly	53

Keep the set alive	54
Practice before mainnet	54
Rotation	55
Offboard	55
Watch the set	56
Paper that survives people	57
Field cards	58
Conclusion	61

ABOUT – SECURITY ALLIANCE

Security Alliance (SEAL) is a nonprofit organization dedicated to securing the future of cryptocurrency. Under the leadership of whitehat hacker **samczsun**, the organization designs security projects and coordinates across the crypto industry so that useful practices actually get used.

Learn more at securityalliance.org

ABOUT – THE RED GUILD

A nonprofit that treats security as a public good in the Ethereum ecosystem. Research, disclosure, education, and tooling.

Learn more at theredguild.org

About this book

Hi reader! o/

This mini-book is an adaptation of the Multisig for Protocols framework from the Security Frameworks, an initiative by the Security Alliance. It is written for the person who is about to sign, or who is designing the set those people will live in.

A threshold is not a security model. Cryptography already works. Protocol multisigs fail operationally: wrong sizing, one God-mode Safe, skipped verification, a signer who is actually three laptops in the same bag.

This is not, by any means, an extensive guide or expected to be followed to the T. Its intention is to guide and provide hints as to where to apply security. Your threat model still wins.

You can access the contents of this book, and a possibly more updated version, on the Security Frameworks website.

Feel free to suggest an update by contributing directly there.

How to use this book

Three words. If the rest falls out of the bag, keep these:

1. **Separate.** Upgrade, pause, and treasury must not live on one set.
2. **Delay.** A timelock without someone watching is decoration.
3. **Verify.** You authorize an effect, not a hash. Decode it. Safe is the default stack here because that is what most protocol sets actually run. The checks travel. The buttons do not.

If you are joining a set you did not design, read “Who may sign” and “How you join” before you accept. You can refuse a bad set.

If you value these type of resources, please consider supporting them. You can do so by going to our website.

securityalliance.org/donate

What actually fails

A poorly run multisig is theater. The contract did its job. The people did not.

Most protocol losses that involve a “multisig” are not broken signature math. Someone signed the wrong thing. Someone was unavailable when pause was needed. Someone put upgrade, treasury, and pause on the same Safe because it was simpler. Someone copied an address from an explorer. Someone trusted the UI.

You are the control. The hardware wallet is a locked box. It will happily

sign a theft if you confirm the hash without reading the effect.

The failure modes that matter

- **God-mode set.** One Safe owns everything. Compromise once, lose the protocol.
- **Rubber stamps.** Threshold signatures collected, nobody decoded the calldata.
- **Silent delay.** Timelock queued a malicious payload. Nobody was watching, so it executed on schedule.
- **Coupled signers.** Same seed, same laptop, same apartment, same

employer. The M in M-of-N was a story.

- **Stuck set.** N-of-N, or keys lost, or no backup UI when Safe.app is dark.
- **Lookalike Safe.** Anyone can add you as owner of a Safe they created. Dashboard lists lie.

What this book is not

It is not a Safe tutorial. Buttons move. The checks do not: full address, decoded function, simulated effect, independent hash.

It is not Wallet Security. Seed hygiene, hardware setup, and personal 2-of-3 live there.

It is not the travel book or the physical-security book. One sentence each, then go read those:

Do not travel with seed material. A threshold must still hold if one signer is coerced.

The job

If you only remember one line from this chapter:

You are not confirming a hash. You are authorizing an on-chain effect.

Map the powers

Do not pick a threshold yet. First write down what the protocol can actually do to itself.

If nobody can name who holds `upgradeTo`, `owner`, `pauser`, or the `treasury`, you do not have a multisig design. You have a spreadsheet of addresses.

Access control inventory

For every privileged role, record:

- Role name on-chain.

- Which address or Safe holds it.
- Which contract it controls.
- What it can do, in functions, not slogans.
- What already constrains it: timelock, parameter bounds, governance.
- Blast radius if that role is evil, or silent.

Keep it living. New deploy, new role, update the page. This inventory is what later chapters call “the set.”

Two questions

For each role, answer both:

1. What happens if this set is compromised?
2. What happens if this set cannot sign for a week?

Upgrade gone wrong can rewrite the protocol. Pause gone missing can leave it draining. Treasury gone missing can freeze payroll and grants. Those are different failures. They want different sets.

Impact, not a dollar table

The website uses dollar bands as a thinking aid. They are examples, not policy. Err toward slower and higher when you are between two labels.

Think in four words:

- **Low.** Alternative path exists. Protocol barely notices.
- **Medium.** Ugly week. Recoverable.
- **High.** Users feel it. Recovery is a project.
- **Critical.** Protocol-wide failure or a treasury you cannot shrug off.

Then ask how fast it must move: routine (a day or two), time-sensitive (hours), emergency (minutes). A critical emergency pause is not designed like a critical upgrade. One wants speed and a tiny blast radius. The other wants slowness and a high threshold.

Write the decision down

Purpose, assets, how often it signs, how fast it must respond, who is in the room. If that page does not exist, the rest of this book is improvisation.

Split the sets

Do not put all admin privileges in one Safe. If that one set is compromised, everything is lost. Separate functional areas so one bad day cannot upgrade, pause, and drain treasury in one shot.

Why split

Blast radius. Right-sized security. The people who understand an upgrade are not always the people who should move funds. Pause needs to be reachable at 3am. Upgrade should be boring and slow.

No single set should have both pause and upgrade unless the upgrade path is gated by a timelock you actually monitor.

Shapes, as examples

These are starting points, not law. Match them to the inventory you just wrote.

- **Protocol upgrades.** High threshold (think 7-of-9 or similar). Long delay, a week or more. Users may need time to leave.
- **Operational parameters.** Moderate threshold. Medium delay, a couple

of days. Parameter bounds on-chain beat “please be careful.”

- **Emergency pause.** Lower threshold (think 2-of-4). No timelock. Constrained to pause-only. This set must not upgrade or move funds.
- **Treasury.** High threshold, different people from upgrade. Split large pots across wallets. Hot wallet is an intermediary, not the vault.
- **Specialized subsystem.** Oracle, bridge, whatever domain experts you actually have. Moderate, medium delay.

Treasury extra

If the Safe holds DAO assets, plan a governance rescue path (the framework names the Allowance module for this). Cold storage holds the bulk. Hot wallet at least 2-of-3. High-value sets want at least one required signer outside the organization.

Seldom-used wallets get longer delays. Any activity on them should wake the whole team.

On-chain beats policy

Whitelist destinations. Bound parameters. Revert on broken invariants. A compromised set that cannot call

upgradeTo or send to a random address is a bad week, not a funeral.

Do not install extra Safe modules or guards without a review you would defend in public. Wrong module can brick the set.

Delay

Build slowness in on purpose. Speed is a risk. Forced urgency is how attackers push a signing ceremony past verification.

Without a timelock, once you have threshold signatures the change lands immediately. There is no window to notice. Legitimate operations can wait. If they cannot, they probably belong on the pause-only set, not the upgrade set.

Tiered delays

Not every change is an upgrade.

- **Long (7–14+ days).** Upgrades, token changes, anything users should be able to exit.
- **Medium (48–72 hours).** Fees, oracles, validator sets. Time to review.
- **Short (hours to a day).** Routine knobs inside bounds you already enforced on-chain.
- **None.** Pause-only. Nothing else.

A default of a few days is a reasonable floor for things that are not emergencies. Even an hour with someone watching beats zero.

The window is the control

A timelock with nobody watching is decoration. The malicious payload just waits, then executes.

Watch queue, execute, and cancel. Alerts must include decoded details: target, function, parameters. Not “something happened.”

At least one watch is run by someone outside the core team, so an insider cannot mute every channel. Use more than one channel. If the Slack is owned, email still rings.

Make the queue visible to the community if you can. Extra eyeballs are part of the delay.

Simulate the right thing

When you sign a timelock queue, the simulation shows the staging event, not the eventual effect. Inspect the staged payload. Check who can cancel and who can execute.

After it lands, verify the on-chain result against what you thought you queued. Local review and broadcast payload are not guaranteed to be the same thing.

Emergency bypass

If you can skip the delay, that path must be pause-only. Never upgrade. Never move funds.

A smaller veto group can cancel. Bypassing a timelock, if you allow it at all, should need the normal quorum plus extra signers, not fewer.

Who may sign

The security of the set is the independence of the signers. M-of-N is a story until the M cannot be collected from one apartment, one employer, or one compromised seed.

The floor

- At least three signers. Never N-of-N: lose one key, brick the set.
- Hardware wallets. Browser-extension keys are not signers, they are incidents waiting.
- Around half the set as threshold,

then raise it for upgrades and large treasuries.

- Dedicated address per Safe. Same device is fine if you use a different index and label it. Same seed still couples them: seed theft hits every index.

High-value sets want an external required signer: security partner, advisor, someone who does not share your Slack.

Independence

Spread people, orgs, devices, manufacturers, geography. Mix roles: not five copies of the same founder. Enough

technical signers that someone in the quorum can actually read calldata.

Do not store two signer keys on one laptop and call it distribution.

Proposer is not signer

On Safe, authorize a delegated proposer that cannot sign. Hash tools read the Safe API. The API only has the transaction after it is proposed. Without a proposer, the first signer is flying blind.

All signers sign only. A non-signer executes. Never “sign and execute” for the last signature. Verification tools do not give you the hashes you think they

do for that combined action.

Nested Safes

A Safe as owner of another Safe. Sometimes an entity wants 1-of-N inside their nest. The nested set can then rotate its own owners without the parent's consent.

Usually do not do this on a protocol set unless you can explain why in writing.

You can refuse

If the set is God-mode, N-of-N, no hardware, no delay on upgrades, or “we all use the same laptop,” do not

join. Being signer number six on a theater quorum does not make you safer. It makes you liable.

How you join

Join with a dedicated hardware-wallet address and prove you own it before anyone relies on you to sign.

Fresh address

New index, labeled, for this Safe only. Two reasons: your wallet then only shows this set's traffic, and you are less likely to sign the wrong request. Remember the seed still ties the indexes together.

Prove ownership

Sign a message that names you (or your entity), the set, and the address. Entity affiliation is fine. The point is accountability, not doxing.

Publish it, or hand it to the admin to file with the inventory. Other signers verify the signature against that address before they add you.

Do this on a tool you already trust, from a bookmark, not from a chat link.

Out of band

Adding or replacing a signer is not a chat-message job. Video plus signed message. If the old key is gone, iden-

tity is video, known social account, or whatever you pre-agreed. Not a Telegram DM from a new number.

Channels before the first real tx

Dedicated channel for this set. Signal is the usual pick. Backup on a different platform. Emergency sets need paging that actually reaches people at 3am. Test it before you need it.

Do not click transaction links. Type or bookmark.

Bookmark the real Safe

Anyone can create a Safe and add you as owner. It shows up in the dashboard next to the real one. Attackers pick addresses that match the first and last four characters of yours.

Navigate from a bookmark. Check the full address. If a new Safe appears that you did not expect, do not touch it.

Verify before you sign

This is the 1am chapter. If you skip it, the rest of the book was conversation.

Safe is the default here because that is the de-facto protocol wallet. Squads and others want the same job: match an independent hash, decode the effect, refuse what you do not understand. Their screens differ. The job does not.

You are not confirming a hash. You are authorizing an on-chain effect.

Before anything else

Hardware wallet. Separate browser profile for signing. Dedicated machine if you can; your daily laptop is a malware lottery.

Confirm you are on the bookmarked Safe, full address, not the dashboard lookalike.

If the queue item is unclear, do not sign. Ask. Every payload should arrive with a “how to check” note. A message from a co-signer is not a third party.

Two phases (Safe / EVM)

When you add a signature you are usually not sending a transaction. You are signing an EIP-712 message.

- **Domain hash.** Tied to this Safe and network. Stops replay elsewhere.
- **Message hash.** The parameters. This is what the hardware wallet shows.
- **SafeTxHash.** Domain plus message. Nested approvals use this.

Recompute the hash locally from nonce, to, value, data, and the rest. Compare the entire hash on the device, not the first and last characters. They must match.

At least two devices, two channels. At least two signers simulate. Simulations lie: they can show one thing and the chain can do another. Use your brain.

Once M signatures exist, execution is a new on-chain call: `execTransaction`. Decode that `calldata`. Check the inner `to`, `value`, and `data` still match what you approved. Re-verify as if you had never seen it.

Hash is not intent

Matching hashes means you are signing the proposed transaction. It does not tell you what it does. Decode the `calldata`.

EVM calldata starts with a 4-byte selector, then ABI-encoded arguments. 0xa9059cbb is transfer. For MultiSend, decode every inner call. One extra call in a batch is how “just a transfer” becomes an approval.

Checksummed addresses only.

If Safe infrastructure is itself hostile, a CLI that reads the Safe API can agree with the UI and still be wrong. Decode the calldata the tool prints. Interactive mode that does not decode is not a complete check.

Red flags. Stop.

DO NOT SIGN

DELEGATECALL (operation 1) unless the target is a known, audited module.

Unexpected approve or
setApprovalForAll.

MultiSend when you were briefed on a single action.

Unknown selector. Unverified target contract. Wrong recipient, amount, or destination.

Decoded function and simulation disagree. The Bybit theft looked benign in the decoder while the unverified contract did something else. Reconcile both, or walk away.

Address book

Never copy addresses from explorer history, chat, or email. Poisoning is real. Copy from a maintained address book or the custody UI.

New recipient: tiny transfer both ways before any real amount. Large send: read the address character by character on a video call. If they “changed wallets,” treat it as a first-time recipient again.

After you sign

Say it: “checked, signed, X more required.” Last signer executes, or says

they cannot. If someone else executes later, they re-check from scratch.

Do not all use the same verification tool.

Solana / Squads

Architecture and what the device displays are different. The job is the same: independent recompute, decode the instruction, do not trust the hosted UI alone. Use that stack's current verification page when you are on it. Do not improvise a Safe ritual on a Squads payload.

When it breaks

Pre-agree the steps. Do not invent process under pressure.

Key compromise, first 30 minutes

1. Stop non-emergency transactions.
2. Notify on every channel, including the backup. Use the template later in this chapter.
3. Scope: which keys, which devices.
4. Escalate to whoever your security contact is.

5. Write the timeline down while you still remember it.

Then isolate devices, new hardware, new seed, rotate the signer on-chain, confirm the new setup before anything resumes.

Lost key is not compromise. Try the backup device. Call the team on the backup channel. Do not panic. You will need video (or the pre-agreed identity check) because you cannot sign a proof with the missing key.

Channel takeover

If Signal, Telegram, or Discord is gone, assume recent messages are hostile.

Alert on the other platform. Cancel anything that started in the compromised thread. Video the person. Do not resume until the rest of the set says the account is clean.

Backup UI

Practice this on a quiet day. Backup infrastructure you first touch during an incident is not infrastructure.

For Safe: Eternal Safe (self-host or IPFS), plus your own RPC. For Squads: the public client, same idea. Self-hosted Safe UI if you already run one.

Treat emergency UIs as extra-untrusted. Verify harder, not less.

Have more than one RPC provider.
Public RPC is a fair-weather tool.

Pause

The emergency set executes pause.
Streamlined verification, not skipped
verification. Geographic spread so
someone is awake. Afterward, write
down what you signed and why.

Drills

Paging test often enough that it is
boring. Full simulation once a year
with the actual signers. Measure time
to threshold. Update the runbook

when it fails, not in the postmortem of a real pause.

Say it clearly

Something like:

```
[URGENT] Multisig incident -  
[name]  
address / class / type / when /  
who  
what happened, what we stopped,  
who can still sign, which  
channel is live
```

Keep the set alive

A set that nobody can rotate is a set that will fail closed or fail open. Both are bad.

Practice before mainnet

Create, sign, rotate an owner on test-net. Then a tiny transfer on mainnet with every signer, on the real channels. If someone cannot sign the test, they cannot sign the pause.

Rotation

Signers can change. Document it. Do not shrink N or M without a written reason.

If the old key still works, prove the new address with the old one, then follow the same ownership proof as joining.

If the old key is gone, identity first, then add.

Admin changes go through two channels, not one chat.

Offboard

Leaving is not logging out. Schedule the removal, execute it, confirm your

address is gone on-chain, update the inventory, leave the channels, delete local copies, hand over anything still pending.

Agree the clock in advance. Emergency sets may need removal in a couple of days. Others can wait a week or two. Residual signer rights are still signer rights.

Watch the set

Alert on proposed transactions, new signatures, owner changes. Watch the signer EOAs too: activity that is not a multisig operation is a smell.

Monitoring channels should be hard to

mute quietly. Tampering with the alert channel should itself alert.

Paper that survives people

Purpose, rules, Safe address, signer addresses, who proposes, who executes, backup UI, backup RPC, who to page. If that page lives in one person's head, they are a single point of failure wearing documentation as a costume.

Field cards

These are prompts, not a binder. Customize addresses and channels before the first real operation.

BEFORE YOU SIGN

Bookmarked Safe, full address.

Hardware wallet, dedicated profile.

“How to check” arrived with the payload.

Independent hash matches the device.

Entire hash.

Calldata decoded: function, target, params.

Simulation and decoder agree. Still use judgment.

No DELEGATECALL unless the module is known.

“Checked, signed, X more.”

POISONED SAFE / ADDRESS BOOK

Dashboard can list a Safe you never created.
First four and last four characters matching
is not identity.

Navigate from a bookmark.

Never paste from explorer history or chat.

New recipient: tiny transfer both ways.

Large send: read the address on a video call.

COMPROMISE, 30 MINUTES

Stop non-emergency transactions.

Notify every channel, including backup.

Scope the keys. Isolate devices.

New hardware, then rotate on-chain.

Lost key is not compromise. Still call the
team.

PAUSE-ONLY

The fast set may pause.

It may not upgrade.

It may not move funds.

Timelock bypass, if it exists, is this set, not a quieter path to upgradeTo.

Drill it. Backup UI already practiced.

Conclusion

Operational security for a protocol set is not about collecting M signatures. It is about making it harder for one bad day to rewrite the contracts, and easier to pause when you must.

Separate the powers. Delay what can wait. Verify the effect, not the chrome.

You can refuse to join a theater quorum. That is part of the job.

When you rotate, offboard, or survive an incident, write down what changed. The next signer should not have to reverse-engineer your memory.

Keep the footprint of each key small, act on purpose, and do not let the UI become the authority.

The live pages move. This book is a companion. If they disagree, check the date, then think. Do not skip the decode either way.

ACKNOWLEDGMENTS

This pocket book follows the Multisig for Protocols framework and the Wallet Security signing pages. Those were written and reviewed in public: isaac, geoffrey, louis, pablo, dickson, engn33r, pinalikefruit, ElliotFriedman, auditware, Coinspect, and the W3OSC work that landed in Frameworks.

Wallet Security is stewarded by Piña. SEAL Certs and related multisig ops material by Dickson Wu and Isaac Patka.

Thanks to SEAL for the pocket format this sits in, to The Red Guild, and to everyone who keeps the live pages honest when a UI changes.

@ 2026 Security Alliance

LICENSE

This edition is an adaptation of the open-source Security Alliance Security Frameworks. Framework text remains copyright Security Alliance and contributors. The pocket arrangement is Creative Commons Attribution-ShareAlike 4.0.

<https://creativecommons.org/licenses/by-sa/4.0/>

Branch develop

Commit 313d3051cab0

Gentium Book is SIL OFL 1.1. Latin Modern is GUST Font License.

Pocket edition – 2026

Published by Security Alliance (SEAL)